大家好，我是 Guide。在 AI 辅助求职场景中，模拟面试需要解决如何**感知候选人背景、控制面试节奏以及产出专业评估报告**等问题。本项目利用 Spring AI、Redis 和状态机理论构建了一套模拟面试系统。

> **与《Spring AI 与大模型集成》的关系**：上篇文章聚焦于技术实现细节（ChatClient、Prompt 管理、RAG 原理等），本文则从**业务架构与系统设计**角度，探讨模拟面试功能的完整设计方案。

系统架构总览

模拟面试功能需要解决三个核心问题：

问题域	挑战	解决方案
个性化出题	如何基于简历生成针对性问题？	AI 问题生成 + 追问机制
交互体验	如何保证长流程的流畅性？	双层缓存 + 断点续答
专业评估	如何给出有价值的反馈？	分批评估 + 二次总结

系统分层架构：

```
┌──────────────────────────────────────────────────────────────────────┐
│                          表现层 (Presentation)                        │
│   ┌──────────────┐   ┌──────────────┐   ┌──────────────┐            │
│   │  面试列表页  │   │  面试详情页  │   │  评估报告页  │            │
│   └──────────────┘   └──────────────┘   └──────────────┘            │
├──────────────────────────────────────────────────────────────────────┤
│                          应用层 (Application)                         │
│   ┌──────────────┐   ┌──────────────┐   ┌──────────────┐            │
│   │ 会话管理服务 │   │ 问题生成服务 │   │ 评估报告服务 │            │
│   └──────────────┘   └──────────────┘   └──────────────┘            │
├──────────────────────────────────────────────────────────────────────┤
│                          领域层 (Domain)                              │
│   ┌──────────────┐   ┌──────────────┐   ┌──────────────┐            │
│   │  会话状态机  │   │  追问策略    │   │  评分聚合器  │            │
│   └──────────────┘   └──────────────┘   └──────────────┘            │
├──────────────────────────────────────────────────────────────────────┤
│                          基础设施层 (Infrastructure)                  │
│   ┌──────────────┐   ┌──────────────┐   ┌──────────────┐            │
│   │ Redis 缓存   │   │ PostgreSQL   │   │ Spring AI    │            │
│   └──────────────┘   └──────────────┘   └──────────────┘            │
└──────────────────────────────────────────────────────────────────────┘
```

有状态会话管理

从 Stateless 到 Stateful

大语言模型（LLM）本身是无状态（Stateless）的，每一轮交互默认不保留历史。但在面试场景中，系统必须追踪以下上下文：

- 候选人的简历背景；
- 已提问数量与当前进度；
- 历史回答的逻辑连贯性。

本项目采用 **Stateful Session** 模式，通过外部存储（Redis + PostgreSQL）维护会话快照，避免在每轮对话中重复传输冗长的简历文本，由业务层精准控制状态流转。

有限状态机（FSM）的应用

面试生命周期被抽象为标准的有限状态机，以确保操作的合法性与流程的确定性。

状态	说明	触发动作
CREATED	题目已根据简历生成	用户开始面试
IN_PROGRESS	用户正在提交各题答案	用户提交所有答案/交卷
COMPLETED	答案采集完毕，评估任务进行中	异步评估任务完成
EVALUATED	报告生成完毕，会话结束	查看报告

状态转换的业务规则

状态枚举定义在 `InterviewSessionDTO` 中（`app/src/main/java/interview/guide/modules/interview/model/InterviewSessionDTO.java`）：

```
public enum SessionStatus {
    CREATED,        // 会话已创建
    IN_PROGRESS,    // 面试进行中
    COMPLETED,      // 面试已完成
    EVALUATED       // 已生成评估报告
}
```

关键业务规则（在 `InterviewSessionService` 中实现）：

- **CREATED → IN_PROGRESS**：只能转换一次，记录面试开始时间
- **IN_PROGRESS → IN_PROGRESS**：允许用户跳题、修改答案，每题独立提交
- **IN_PROGRESS → COMPLETED**：用户主动交卷或全部题目答完
- **COMPLETED → EVALUATED**：由异步任务触发，不可逆

数据模型设计

核心实体关系

```
┌──────────────────┐        ┌──────────────────────────────┐
│   Resume         │ 1──N │      InterviewSession         │
│   (简历)         │        │          (会话表)             │
├──────────────────┤        ├──────────────────────────────┤
│ id               │        │ id                            │
│ ...              │        │ sessionId (唯一)              │
└──────────────────┘        │ resumeId                      │
                            │ status / evaluateStatus       │
                            │ totalQuestions                │
                            │ currentQuestionIndex          │
                            │ questionsJson  ← 题目快照     │
                            │ overallScore / overallFeedback│
                            │ strengthsJson / improvementsJson│
                            └──────────────┬───────────────┘
                                           │ 1──N
                            ┌──────────────▼───────────────┐
                            │      InterviewAnswer          │
                            │          (答案表)             │
                            ├──────────────────────────────┤
                            │ id                            │
                            │ session_id                    │
                            │ question_index                │
                            │ question / category           │
                            │ userAnswer / score / feedback│
                            │ referenceAnswer / keyPoints   │
                            │ 唯一约束: (session_id,question_index) │
                            └──────────────────────────────┘
```

> 说明：当前实现中，`isFollowUp` 与 `parentQuestionIndex` 存在于 `InterviewQuestionDTO`（序列化进 `questionsJson`），而不是单独的数据库列。

追问的数据模型设计

追问系统采用**线性展开**策略，将树状的问题结构（主问题 + 追问）扁平化为线性列表。

设计优势：

- **简化前端渲染**：无需递归组件，顺序渲染即可

- **统一交互逻辑**：主问题和追问的答题、跳过逻辑一致
- **便于评估聚合**：所有问题平铺后，分批评估逻辑更简单

数据示例：

```
[
  {
    "questionIndex": 0,
    "question": "MySQL 的索引有哪些类型？",
    "category": "MySQL",
    "isFollowUp": false,
    "parentQuestionIndex": null
  },
  {
    "questionIndex": 1,
    "question": "基于上一题，请解释 B+ 树索引的查找过程。",
    "category": "MySQL（追问1）",
    "isFollowUp": true,
    "parentQuestionIndex": 0
  },
  {
    "questionIndex": 2,
    "question": "Redis 支持哪些数据结构？",
    "category": "Redis",
    "isFollowUp": false,
    "parentQuestionIndex": null
  }
]
```

问题生成策略

题型分布算法

问题生成采用**加权分布算法**，确保技术栈覆盖均衡。实现在 `InterviewQuestionService` 类中（`app/src/main/java/interview/guide/modules/interview/service/InterviewQuestionService.java`）：

```java
@Service
public class InterviewQuestionService {

    // 问题类型权重分配（按优先级）
    private static final double PROJECT_RATIO = 0.20;      // 20% 项目经历
    private static final double MYSQL_RATIO = 0.20;        // 20% MySQL
    private static final double REDIS_RATIO = 0.20;        // 20% Redis
    private static final double JAVA_BASIC_RATIO = 0.10;   // 10% Java基础
    private static final double JAVA_COLLECTION_RATIO = 0.10; // 10% 集合
    private static final double JAVA_CONCURRENT_RATIO = 0.10; // 10% 并发
    // Spring/SpringBoot 数量由总题数减去其余类型得出（余额调整），无单独比例常量

    private QuestionDistribution calculateDistribution(int total) {
        int project = Math.max(1, (int) Math.round(total * PROJECT_RATIO));
        int mysql = Math.max(1, (int) Math.round(total * MYSQL_RATIO));
        int redis = Math.max(1, (int) Math.round(total * REDIS_RATIO));
        int javaBasic = Math.max(1, (int) Math.round(total * JAVA_BASIC_RATIO));
        int javaCollection = (int) Math.round(total * JAVA_COLLECTION_RATIO);
```

```
        int javaConcurrent = (int) Math.round(total * JAVA_CONCURRENT_RATIO);
        int spring = total - project - mysql - redis - javaBasic -
javaCollection - javaConcurrent;
        spring = Math.max(0, spring);

        return new QuestionDistribution(project, mysql, redis,
            javaBasic, javaCollection, javaConcurrent, spring);
    }

    private record QuestionDistribution(
        int project, int mysql, int redis,
        int javaBasic, int javaCollection, int javaConcurrent, int spring
    ) {}
}
```

追问生成的业务规则

追问数量配置:

追问数量	适用场景	预估总题数（10主题）
0	快速摸底、时间紧张	10 题
1	标准面试、平衡深度与广度	20 题
2	深度技术考察、专家面	30 题

追问的生成策略:

1. **关联性约束**：追问必须与主问题相关，可基于回答内容进一步挖掘
2. **梯度递进**：追问应比主问题更具深度或要求实际场景
3. **AI 生成为主，模板兜底**：优先使用 AI 生成个性化追问，失败时使用预设模板

温度参数的应用场景:

温度（Temperature）是调节模型输出随机性与确定性的参数。在面试场景中，不同阶段需要不同的温度设置：

场景	建议温度	说明
问题生成	0.3-0.5	较低温度确保问题规范、专业，避免生成过于刁钻或偏离主题的问题
追问生成	0.5-0.7	适中温度允许一定灵活性，根据用户回答生成针对性追问
评估打分	0.1-0.3	低温度保证评分标准一致、逻辑严谨
评语生成	0.4-0.6	平衡专业性与个性化，生成有针对性的改进建议

> **注**：温度范围通常为 0.0-1.0。0.0 表示完全确定性（每次输入相同输出相同），1.0 表示高度随机。面试场景建议使用 0.2-0.6 的中等偏低温度。

兜底追问模板（在 `InterviewQuestionService.buildDefaultFollowUp()` 中实现）：

```
private String buildDefaultFollowUp(String mainQuestion, int order) {
    if (order == 1) {
        return "基于\u201C" + mainQuestion + "\u201D，请结合你亲自做过的一个真实场景展开说明。";
    }
    return "基于\u201C" + mainQuestion + "\u201D，如果线上出现异常，你会如何定位并给出修复方案？";
}
```

历史问题去重机制

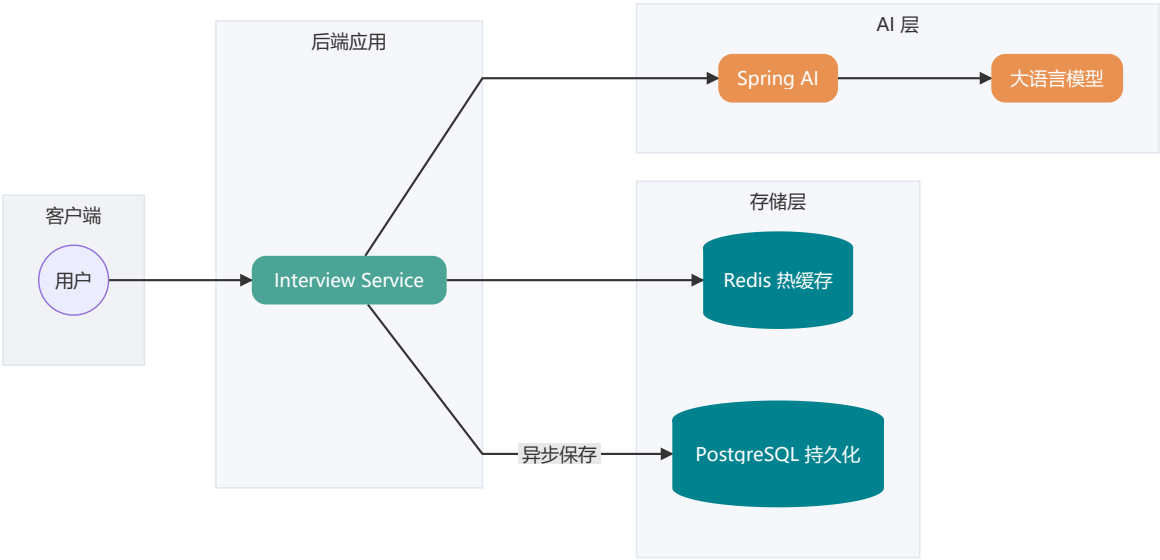
为避免用户多次面试时遇到重复问题，系统实现了**历史问题感知**。实现在 `InterviewQuestionService.generateQuestions()` 方法中：

```
// InterviewSessionService#createSession（当 request.resumeId() != null 时）
List<String> historicalQuestions =
persistenceService.getHistoricalQuestionsByResumeId(request.resumeId());
List<InterviewQuestionDTO> questions = questionService.generateQuestions(
    request.resumeText(),
    request.questionCount(),
    historicalQuestions
);

// InterviewPersistenceService#getHistoricalQuestionsByResumeId
List<InterviewSessionEntity> sessions =
sessionRepository.findTop10ByResumeIdOrderByCreatedAtDesc(resumeId);
return sessions.stream()
    .map(InterviewSessionEntity::getQuestionsJson)
    .filter(json -> json != null && !json.isEmpty())
    .flatMap(json -> {
        List<InterviewQuestionDTO> questions = objectMapper.readValue(json,
            new TypeReference<List<InterviewQuestionDTO>>() {});
        return questions.stream()
            .filter(q -> !q.isFollowUp()) // 排除追问，只保留主问题
            .map(InterviewQuestionDTO::question);
    })
    .distinct()
    .limit(30) // 只保留最近 30 条主问题
    .toList();
```

存储方案：双层缓存与断点续答

面试过程属于长耗时、高频交互场景，系统设计了热冷分离的存储策略以平衡响应速度与可靠性。



Redis 热缓存

面试进行中的中间状态（如已填写的答案、当前索引 `currentIndex`）实时暂存在 Redis 中。

- **Redisson 序列化**：将会话对象（`CachedSession`）序列化为 JSON 存储。
- **性能表现**：题目切换与答案暂存均在内存中完成，在本地网络环境下 p99 延迟约 **< 10ms**（基于 Redis 单机部署测试）。

存储策略与一致性权衡

当状态发生关键变更（如初始化、交卷）时，系统采用 **Write-Through with Degradation** 策略：

- **当前实现**：
  1. 数据优先写入 Redis（快速响应用户）
  2. 同步尝试写入 PostgreSQL（尽力而为）
  3. DB 写入失败时记录 WARN 日志，不阻塞主流程

权衡分析：

维度	本实现	Write-Through（强一致）	Write-Behind（异步回写）
延迟	很低（Redis + DB 尽力写）	中（需等待 DB 事务完成）	最低（只写 Redis）
数据安全	较高（DB 同步写入）	最高（事务保证）	较低（依赖刷盘任务）
一致性窗口	极短（DB 写入失败时）	无窗口	取决于刷盘间隔
DB 压力	中	高	低（批量写入）

失败处理：

- **Redis 写入失败**：操作失败，返回错误给用户
- **DB 写入失败**：记录 WARN 日志，主流程继续（用户无感知）
- **Redis 宕机**：降级直连 DB，响应延迟增加至 50-100ms
- **缓存数据丢失**：从 DB 全量重建，用户无感知
- **DB 与 Redis 不一致**：以 DB 为准（DB 是持久化真相源）

断点续答（Session Resume）：确保用户中途异常退出后，下次进入能从数据库恢复会话并同步至缓存，实现进度的无缝衔接。

恢复流程：

1. 用户再次进入时，系统从 PostgreSQL 查询 `IN_PROGRESS` 或 `COMPLETED` 状态的会话
2. 比对 Redis 缓存是否存在，若不存在则从 DB 反序列化重建
3. 恢复至最后一题的索引位置（`currentIndex`），用户可继续答题

缓存键设计

缓存键的设计实现在 `InterviewSessionCache` 类中（`app/src/main/java/interview/guide/infrastructure/redis/InterviewSessionCache.java`）：

```java
@Service
public class InterviewSessionCache {

    /**
     * 缓存键前缀
     */
    private static final String SESSION_KEY_PREFIX = "interview:session:";

    /**
     * 简历ID到会话ID的映射前缀（用于查找未完成会话）
     */
    private static final String RESUME_SESSION_KEY_PREFIX = "interview:resume:";

    /**
     * 会话默认过期时间（24小时）
     */
    private static final Duration SESSION_TTL = Duration.ofHours(24);

    private String buildSessionKey(String sessionId) {
        return SESSION_KEY_PREFIX + sessionId;
    }

    private String buildResumeSessionKey(Long resumeId) {
        return RESUME_SESSION_KEY_PREFIX + resumeId;
    }

    /**
     * 根据简历ID查找未完成的会话ID（断点续答核心方法）
     */
    public Optional<String> findUnfinishedSessionId(Long resumeId) {
        String key = buildResumeSessionKey(resumeId);
        String sessionId = redisService.get(key);
        // ... 验证逻辑
    }
}
```

缓存键格式说明：

键类型	格式	内容	TTL
会话数据	`interview:session:{sessionId}`	`CachedSession` JSON（包含答案、进度、状态）	24 小时
简历映射	`interview:resume:{resumeId}`	会话 ID（用于查找未完成会话）	24 小时

断点续答实现：通过 `findUnfinishedSessionId(resumeId)` 方法，利用简历映射键快速查找用户是否有未完成的会话。

评估报告生成策略

评分体系设计

评估报告采用**多维度评分体系**，全面反映候选人能力：

维度	权重	评分依据	数据来源
整体得分	100%	所有问题的平均分	所有问题评分的算术平均
类别得分	-	按技术栈分类的平均分	MySQL、Redis、Java 等
问题反馈	-	每题的具体评价和改进建议	AI 逐题分析
综合评价	-	整体表现总结	AI 汇总分析
优势列表	-	3-6 条亮点	从问题评估中提取
改进建议	-	3-6 条具体建议	从问题评估中提取

评分聚合算法

评分聚合逻辑实现在 `AnswerEvaluationService.convertToReport()` 方法中（`app/src/main/java/interview/guide/modules/interview/service/AnswerEvaluationService.java`）：

```
private InterviewReportDTO convertToReport(
    String sessionId,
    List<QuestionEvaluationDTO> evaluations,
    List<InterviewQuestionDTO> questions,
    String overallFeedback,
    List<String> strengths,
    List<String> improvements
) {
    // 统计实际回答的问题数量
    long answeredCount = questions.stream()
        .filter(q -> q.userAnswer() != null && !q.userAnswer().isBlank())
        .count();

    // 收集类别分数，并在循环中填充 questionDetails（每题评估）、referenceAnswers
    List<QuestionEvaluation> questionDetails = new ArrayList<>();
    Map<String, List<Integer>> categoryScoresMap = new HashMap<>();
    for (int i = 0; i < questions.size(); i++) {
```

```
        QuestionEvaluationDTO eval = i < evaluations.size() ? evaluations.get(i)
: null;
        InterviewQuestionDTO q = questions.get(i);
        boolean hasAnswer = q.userAnswer() != null && !q.userAnswer().isBlank();
        int score = hasAnswer && eval != null ? eval.score() : 0;

        questionDetails.add(new QuestionEvaluation(
            q.questionIndex(), q.question(), q.category(),
            q.userAnswer(), score,
            eval != null && eval.feedback() != null ? eval.feedback() : "该题未成
功生成评估反馈。"
        ));
        categoryScoresMap
            .computeIfAbsent(q.category(), k -> new ArrayList<>())
            .add(score);
    }

    // 计算各类别平均分
    List<CategoryScore> categoryScores = categoryScoresMap.entrySet().stream()
        .map(e -> new CategoryScore(
            e.getKey(),
            (int)
e.getValue().stream().mapToInt(Integer::intValue).average().orElse(0),
            e.getValue().size()
        ))
        .collect(Collectors.toList());

    // 计算总分：基于 questionDetails 中的得分，若所有问题都未回答则总分为 0
    int overallScore;
    if (answeredCount == 0) {
        overallScore = 0;
    } else {
        overallScore = (int) questionDetails.stream()
            .mapToInt(QuestionEvaluation::score)
            .average()
            .orElse(0);
    }

    return new InterviewReportDTO(...);
}
```

分批评估的业务逻辑

为什么需要分批评估?

问题数	预估 Token 数	风险
10 题	~15,000	单次可处理
20 题	~30,000	接近部分模型上限
30 题	~45,000	超出上下文限制

分批策略（在 `AnswerEvaluationService.evaluateInBatches()` 中实现）：

```
@Service
```

```java
public class AnswerEvaluationService {

    private final int evaluationBatchSize; // 配置项：默认 8

    private List<BatchEvaluationResult> evaluateInBatches(
        String sessionId,
        String resumeSummary,
        List<InterviewQuestionDTO> questions
    ) {
        List<BatchEvaluationResult> results = new ArrayList<>();
        for (int start = 0; start < questions.size(); start +=
evaluationBatchSize) {
            int end = Math.min(start + evaluationBatchSize, questions.size());
            List<InterviewQuestionDTO> batchQuestions = questions.subList(start,
end);

            EvaluationReportDTO report = evaluateBatch(
                sessionId, resumeSummary, batchQuestions, start, end
            );
            results.add(new BatchEvaluationResult(start, end, report));
        }
        return results;
    }

    private record BatchEvaluationResult(
        int startIndex,
        int endIndex,
        EvaluationReportDTO report
    ) {}
}
```

合并批次结果（当二次总结失败时的降级方案）：

失败模式处理：

- **单批次评估失败**：抛出 `BusinessException`，整个评估流程终止
- **二次总结失败**：使用批次聚合结果作为最终输出（降级）
- **所有批次失败**：抛出 `BusinessException`，评估任务标记为 FAILED 状态

> **实现说明**：当前实现中，单批次评估失败会导致整个流程中断。如需更容错的行为，可以在 `evaluateBatch()` 方法中捕获异常并返回空评估，然后在 `mergeQuestionEvaluations()` 中填充默认值。

大规模评估的优化实践

批量评估

长达 10 道题以上的面试评估涉及大量上下文（题目+回答+参考答案），容易超出 Token 限制或因耗时过长导致超时。

- **分片策略**：按 `batch-size` 将题目拆分为多个批次分别评估。
- **两阶段汇总**：各批次生成中间评分后，再调用汇总 Prompt 进行全局优缺点分析。

异步处理

评估任务通常耗时 10 秒以上。系统采用 **Redis Stream** 异步处理模式：

1. 用户交卷后，系统将评估任务发送到 Redis Stream，立即返回任务 ID；
2. **EvaluateStreamConsumer**（虚拟线程）监听 Stream，执行 AI 评估；
3. 评估完成后，更新数据库状态，前端通过轮询获取最终报告。

异步架构优势：

- **解耦**：评估任务与主流程分离，避免阻塞用户请求
- **可扩展**：可增加消费者数量并行处理评估任务
- **可靠性**：Redis Stream 持久化消息，防止任务丢失

幂等性保障：面试答案写入采用 upsert 方式，并通过 `interview_answers` 的唯一约束 `(session_id, question_index)` 保证同一题只保留一条记录。

⚠**** 风险提示：****

虚拟线程虽然解决了线程阻塞问题，但在调用 LLM 供应商（如 OpenAI/Azure）API 时，必须配合 **Resilience4j 的 RateLimiter/Bulkhead**。如果不加限制，大量并发的虚拟线程会瞬间产生数千个 HTTP 请求，导致 LLM API 触发 **429 Too Many Requests** 错误。虚拟线程缓解了服务器 I/O 压力，但却将瓶颈转移到了外部 API 限流策略上，因此实施客户端限流至关重要。

限流配置：本项目采用 **Redis + Lua 滑动时间窗口** 算法实现多维度原子限流。

接口	限流阈值	说明
创建面试会话	5 次/秒	GLOBAL + IP 双维度
提交答案	10 次/秒	GLOBAL 单维度
简历上传分析	5 次/秒	GLOBAL + IP 双维度
简历重新分析	2 次/秒	GLOBAL + IP 双维度
知识库查询	10 次/秒	GLOBAL + IP 双维度
知识库流式查询	5 次/秒	GLOBAL + IP 双维度
知识库上传	3 次/秒	GLOBAL + IP 双维度

实现细节：

- 通过 `@RateLimit` 注解声明式配置限流策略
- 支持 `GLOBAL`、`IP`、`USER` 三种维度组合
- Lua 脚本确保多维度限流的原子性（所有维度都通过才扣减令牌）
- 支持降级方法（`fallback`）配置

核心代码位置（路径分别相对于 `app/src/main/java/interview/guide/` 与 `app/src/main/resources/`）：

- 限流注解：`@RateLimit`（`common/annotation/RateLimit.java`）
- AOP 切面：`common/aspect/RateLimitAspect.java`
- Lua 脚本：`scripts/rate_limit.lua`

失败处理（当前实现口径）：

- AI 调用失败会走业务异常处理，评估流程可降级到分批聚合结果；
- 对结构化输出场景使用统一封装与有限重试，降低 JSON 解析失败带来的不可用风险；
- `spring.ai.retry.max-attempts=1`，优先快速失败，避免无效重试放大延迟。

总结

构建 AI 面试系统的关键不在于堆砌模型参数，而在于**对业务逻辑的建模（状态机）**以及对不确定性（LLM 输出）的结构化封装**。

实践建议：

- **逻辑剥离**：复杂的业务校验和题号计算应由 Java 代码完成，避免模型在计算逻辑上产生幻觉；
- **提示词版本化**：利用 `.st` 模板文件管理提示词，并纳入 Git 版本控制；
- **缓存优先**：在高频交互场景下，Redis 缓存对提升用户体验至关重要。